

py-typedlogic

None

None

None

Table of contents

1. py-typedlogic: Bridging Formal Logic and Typed Python	4
1.1 Key Features	4
1.2 Installation	5
1.3 Define predicates using Pythonic idioms	5
1.4 Specify logical axioms directly in Python	5
1.5 Performing reasoning from within Python	5
2. Concepts	7
2.1 typed-logic: Bridging Formal Logic and Typed Python	7
2.2 Core Concepts	7
2.3 Data Model	10
2.4 Decorators	0
2.5 Generators	0
2.6 Solvers	0
2.7 Models	0
3. Solvers	0
3.1 Solvers	0
3.2 Clingo Integrations	0
3.3 Z3 Integrations	0
3.4 Souffle Integrations	0
3.5 Prover9 Integrations	0
3.6 SnakeLog Integrations	0
3.7 ProbLog Integrations	0
3.8 LLM Integrations	0
4. Compilers	0
4.1 Compilers	0
4.2 YAML Compiler	0
4.3 Prolog Compiler	0
4.4 ProbLog Compiler	0
4.5 TPTP Compiler	0
4.6 FOR Compiler	0
4.7 S-Expression Compiler	0
5. Parsers	0
5.1 Parsers	0
5.2 Python Parser	0
5.3 YAML Parser	0

5.4 DataFrame Parser	0
5.5 Catalog Parser	0
5.6 OWLPython Parser	0
5.7 RDF Parser	0
6. Frameworks	0
6.1 OWL-DL	0
6.2 RDF Integration	0
7. CLI Reference	0
8. typedlogic	0
8.1 convert	0
8.2 Example:	0
8.3 dump	0
8.4 Examples	0
8.5 list-compilers	0
8.6 list-parsers	0
8.7 list-solvers	0
8.8 solve	0
8.9 Examples	0
9. Troubleshooting	0
9.1 Type Checking Errors	0
9.2 Solver Not Finding Solutions	0
9.3 Performance Issues	0
9.4 Integration Issues	0
9.5 Unexpected Logical Results	0
10. Roadmap	0
10.1 Solvers	0
10.2 Framework Integrations	0
10.3 Transformations	0
10.4 Documentation	0
10.5 Current Limitations	0

1. py-typedlogic: Bridging Formal Logic and Typed Python

TypedLogic is a powerful Python package that bridges the gap between formal logic and strongly typed Python code. It allows you to leverage fast logic programming engines like Souffle while specifying your logic in mypy-validated Python code.

links.py run.py stdout Semantics

```
# links.py
from pydantic import BaseModel
from typedlogic import FactMixin, gen2
from typedlogic.decorators import axiom

ID = str

class Link(BaseModel, FactMixin):
    """A link between two entities"""
    source: ID
    target: ID

class Path(BaseModel, FactMixin):
    """An N-hop path between two entities"""
    source: ID
    target: ID
    hops: int

@axiom
def path_from_link(x: ID, y: ID):
    """If there is a link from x to y, there is a path from x to y"""
    assert Link(source=x, target=y) >> Path(source=x, target=y, hops=1)

@axiom
def transitivity(x: ID, y: ID, z: ID, d1: int, d2: int):
    """Transitivity of paths, plus hop counting"""
    assert ((Path(source=x, target=y, hops=d1) & Path(source=y, target=z, hops=d2)) >>
            Path(source=x, target=z, hops=d1+d2))

@axiom
def reflexivity():
    """No paths back to self"""
    assert not any(Path(source=x, target=x, hops=d) for x, d in gen2(ID, int))

from typedlogic.integrations.souffle_solver import SouffleSolver
from links import Link
import links as links

solver = SouffleSolver()
solver.load(links) ## source for definitions and axioms
# Add data
links = [Link(source='CA', target='OR'), Link(source='OR', target='WA')]
for link in links:
    solver.add(link)
model = solver.model()
for fact in model.iter_retrieve("Path"):
    print(fact)
```

Output:

```
Path(source='CA', target='OR', hops=1)
Path(source='OR', target='WA', hops=1)
Path(source='CA', target='WA', hops=2)
```

To convert the Python code to first-order logic, use the CLI:

```
typedlogic convert links.py -t fol
```

Output:

```
∀[x:ID y:ID]. Link(x, y) → Path(x, y, 1)
∀[x:ID y:ID z:ID d1:int d2:int]. Path(x, y, d1) ∧ Path(y, z, d2) → Path(x, z, d1+d2)
¬∃[x:ID d:int]. Path(x, x, d)
```

1.1 Key Features

- Write logical axioms and rules using familiar Python syntax

- Benefit from strong typing and mypy validation
- Integration with multiple solvers and logic engines, including Z3 and Souffle
- Compatible with popular Python validation libraries like Pydantic

1.2 Installation

Install TypedLogic using pip:

```
pip install typedlogic
```

With all extras pre-installed:

```
pip install typedlogic[all]
```

You can also use pipx to run the [CLI](#) without installing the package globally:

```
pipx run typedlogic --help
```

1.3 Define predicates using Pythonic idioms

Inherit from one of the TypedLogic base models to add semantics to your data model. For Pydantic:

```
from typedlogic.integrations.frameworks.pydantic import FactBaseModel

ID = str

class Link(FactBaseModel):
    source: ID
    target: ID

class Path(FactBaseModel):
    source: ID
    target: ID
```

These can be used in the standard way in Python:

```
links = [Link(source='CA', target='OR'), Link(source='OR', target='WA')]
```

1.4 Specify logical axioms directly in Python

Once you have defined your data model predicates, you can specify logical axioms directly in Python:

```
from typedlogic.decorators import axiom

@axiom
def link_implies_path(x: ID, y: ID):
    """For all x, y, if there is a link from x to y, then there is a path from x to y"""
    if Link(source=x, target=y):
        assert Path(source=x, target=y)

@axiom
def transitivity(x: ID, y: ID, z: ID):
    """For all x, y, z, if there is a path from x to y and a path from y to z,
    then there is a path from x to z"""
    if Path(source=x, target=y) and Path(source=y, target=z):
        assert Path(source=x, target=z)
```

1.5 Performing reasoning from within Python

Use any of the existing solvers to perform reasoning:

```
from typedlogic.integrations.snakelogic import SnakeSolver

solver = SnakeSolver()
solver.load("links.py") ## source for definitions and axioms
for link in links:
    solver.add(link)
```

```
model = solver.model()
for fact in model.iter_retrieve("Path"):
    print(fact)
```

outputs:

```
Path(source='CA', target='OR')
Path(source='OR', target='WA')
Path(source='CA', target='WA')
```

2. Concepts

2.1 typed-logic: Bridging Formal Logic and Typed Python

typed-logic is a Python package that allows Python data models to be augmented using formal logical statements, which are then interpreted by *solvers* which can reason over combinations of programs and data allowing for *satisfiability checking*, and the generation of new data.

Currently the solvers supported are:

- [Z3](#)
- [Souffle](#)
- [Clingo](#)
- [Prover9](#)
- [Snakelog](#)
- [ProbLog](#)

With support for more solvers (Vampire, OWL reasoners, etc.) planned.

typed-logic is aimed primarily at software developers and data modelers who are logic-curious, but don't necessarily have a background in formal logic. It is especially aimed at Python developers who like to use lightweight ways of ensuring program and data correctness, such as Pydantic for data and mypy for type checking of programs.

2.1.1 Key Features

- Write logical axioms and rules using familiar Python syntax
- Benefit from strong typing and mypy validation
- Seamless integration with logic programming engines
- Support for various solvers, including Z3 and Souffle
- Compatible with popular Python libraries like Pydantic

2.1.2 Why TypedLogic?

TypedLogic combines the best of both worlds: the expressiveness and familiarity of Python with the power of formal logic and fast logic programming engines. This unique approach allows developers to:

1. Write more maintainable and less error-prone logical rules
2. Catch type-related errors early in the development process
3. Seamlessly integrate logical reasoning into existing Python projects
4. Leverage the performance of specialized logic engines without sacrificing the Python ecosystem

Get started with TypedLogic and experience a new way of combining logic programming with strongly typed Python!

2.2 Core Concepts

TypedLogic is built around several core concepts that blend logical programming with typed Python. Understanding these concepts is crucial for effectively using the library.

2.2.1 Use Python idioms to define your data structures

Facts are the basic units of information in TypedLogic. They are represented as Python classes that inherit from both `pydantic.BaseModel` and `FactMixin`. This approach allows you to define strongly-typed facts with automatic validation.

Example:

```
from pydantic import BaseModel
from typedlogic import FactMixin

PersonID = str
PetID = str

class Person(BaseModel, FactMixin):
    name: PersonID

class PersonAge(BaseModel, FactMixin):
    name: PersonID
    age: int

class Pet(BaseModel, FactMixin):
    name: PetID

class PetSpecies(BaseModel, FactMixin):
    name: PetID
    species: str

class OwnsPet(BaseModel, FactMixin):
    person: PersonID
    pet: PetID
```

Note in many logic frameworks these definitions don't have a direct translation, but in sorted logics, these may correspond to [predicate definitions](#).

2.2.2 Axioms

Axioms are logical rules or statements that define relationships between facts. In TypedLogic, axioms are defined using Python functions [decorated](#) with `@axiom`.

Example:

```
from typedlogic.decorators import axiom

@axiom
def constraints(person: PersonID, pet: PetID):
    if OwnsPet(person=person, pet=pet):
        assert Person(name=person) and Pet(name=pet)
```

Note that the Python code in the function definition is **never executed**. The code must be in a [subset of python](#) that is translated to logic statements.

You can also derive new facts from axioms:

```
class SameOwner(BaseModel, FactMixin):
    pet1: PetID
    pet2: PetID

from typedlogic.decorators import axiom

@axiom
def entail_same_owner(person: PersonID, pet1: PetID, pet2: PetID):
    if OwnsPet(person=person, pet=pet1) and OwnsPet(person=person, pet=pet2):
        assert SameOwner(pet1=pet1, pet2=pet2)
```

2.2.3 Generators

Generators like `gen1`, `gen2`, etc., are used within axioms to create typed placeholders for variables. They help maintain type safety while defining logical rules.

You can use generators in combination with Python `all` and `any` functions to express quantified sentences:

```
@axiom
def entail_same_owner():
    assert all(SameOwner(pet1=pet1, pet2=pet2)
```

```
for person, pet1, pet2 in gen3(PersonID, PetID, PetID)
if OwnsPet(person=person, pet=pet1) and OwnsPet(person=person, pet=pet2))
```

See [Generators](#) for more information.

2.2.4 Solvers

TypedLogic supports multiple [solvers](#), including Z3 and Souffle. Solvers are responsible for reasoning over the facts and axioms to derive new information or check for consistency.

Example (using the [Z3 solver](#)):

```
from typedlogic.integrations.solvers.z3 import Z3Solver

solver = Z3Solver()
solver.add(theory)
result = solver.check()
```

2.2.5 Theories

A [Theory](#) in TypedLogic is a collection of predicate definitions, facts, and axioms. It represents a complete knowledge base that can be reasoned over.

Example:

```
from typedlogic import Theory

theory = Theory(
    name="family_relationships",
    predicate_definitions=[...],
    sentence_groups=[...],
)
```

2.3 Data Model

Data model for the typed-logic framework.

Overview

This module defines the core classes and structures used to represent logical constructs such as sentences, terms, predicates, and theories. It is based on the Common Logic Interchange Format (CLIF) and the Common Logic Standard (CL), with additions to make working with simple type systems easier.

Logical axioms are called [sentences](#) which organized into [theories](#)., which can be loaded into a [solver](#).

While one of the goals of typed-logic is to be able to write logic intuitively in Python, this data model is independent of the mapping from the Python language to the logic language; it can be used independently of the python syntax.

Here is an example:

```
>>> from typedlogic import Term, Forall, Implies
>>> x = Variable('x')
>>> y = Variable('y')
>>> pdef = PredicateDefinition(predicate='FriendOf',
...                             arguments={'x': 'str', 'y': 'str'}),
...
>>> theory = Theory(
...     name="My theory",
...     predicate_definitions=[pdef],
... )
>>> s = Forall([x, y],
...             Implies(Term('friend_of', x, y),
...                     Term('friend_of', y, x)))
>>> theory.add(s)
```

Classes

2.3.1 PredicateDefinition `dataclass`

Defines the name and arguments of a predicate.

Example:

```
>>> pdef = PredicateDefinition(predicate='FriendOf',
...                             arguments={'x': 'str', 'y': 'str'})
...
```

The arguments are mappings between variable names and types. You can use either base types (e.g. 'str', 'int', 'float') or custom types.

Custom types should be defined in the theory's `type_definitions` attribute.

```
>>> pdef = PredicateDefinition(predicate='FriendOf',
...                             arguments={'x': 'Person', 'y': 'Person'})
...
>>> theory = Theory(
...     name="My theory",
...     type_definitions={'Person': 'str'},
...     predicate_definitions=[pdef],
... )
```

Model:

```
classDiagram
class PredicateDefinition {
+String predicate
+Dict arguments
+String description
+Dict metadata
}
PredicateDefinition --> "*" PredicateDefinition : parents
```

Source code in `src/typedlogic/datamodel.py`

```

49 @dataclass
50 class PredicateDefinition:
51     """
52     Defines the name and arguments of a predicate.
53
54     Example:
55
56     >>> pdef = PredicateDefinition(predicate='FriendOf',
57     ...                             arguments={'x': 'str', 'y': 'str'})
58
59     The arguments are mappings between variable names and types.
60     You can use either base types (e.g. 'str', 'int', 'float') or custom types.
61
62     Custom types should be defined in the theory's 'type_definitions' attribute.
63
64     >>> pdef = PredicateDefinition(predicate='FriendOf',
65     ...                             arguments={'x': 'Person', 'y': 'Person'})
66
67     >>> theory = Theory(
68     ...     name="My theory",
69     ...     type_definitions={'Person': 'str'},
70     ...     predicate_definitions=[pdef],
71     ... )
72
73     Model:
74
75     ```mermaid
76     classDiagram
77     class PredicateDefinition {
78     +String predicate
79     +Dict arguments
80     +String description
81     +Dict metadata
82     }
83     PredicateDefinition --> "1" PredicateDefinition : parents
84     """
85
86     predicate: str
87     arguments: Dict[str, str]
88     description: Optional[str] = None
89     metadata: Optional[Dict[str, Any]] = None
90     parents: Optional[List[str]] = None
91     python_class: Optional[Type] = None
92
93     def argument_base_type(self, arg: str) -> str:
94         typ = self.arguments[arg]
95         try:
96             import pydantic
97
98             if isinstance(typ, pydantic.fields.FieldInfo):
99                 typ = typ.annotation
100         except ImportError:
101             pass
102         return str(typ)
103
104     @classmethod
105     def from_class(cls, python_class: Type) -> "PredicateDefinition":
106         """
107         Create a predicate definition from a python class
108
109         :param predicate_class:
110         :return:
111         """
112         return PredicateDefinition(
113             predicate=python_class.__name__,
114             arguments={k: v for k, v in python_class.__annotations__.items()},
115         )
116

```

`from_class(python_class)` classmethod

Create a predicate definition from a python class

Parameters:

Name	Type	Description	Default
<code>predicate_class</code>			<i>required</i>

Returns:

Type	Description
<code>PredicateDefinition</code>	

Source code in `src/typedlogic/datamodel.py`

```

105 @classmethod
106 def from_class(cls, python_class: Type) -> "PredicateDefinition":
107     """
108     Create a predicate definition from a python class
109     :param predicate_class:
110     :return:
111     """
112     return PredicateDefinition(
113         predicate=python_class.__name__,
114         arguments={k: v for k, v in python_class.__annotations__.items()},
115     )
116

```

2.3.2 Variable `dataclass`

A variable in a logical sentence.

```

>>> x = Variable('x')
>>> y = Variable('y')
>>> s = Forall([x, y],
...           Implies(Term('friend_of', x, y),
...                   Term('friend_of', y, x)))

```

Variables can have domains (types) specified:

```

>>> x = Variable('x', domain='str')
>>> y = Variable('y', domain='str')
>>> z = Variable('z', domain='int')
>>> xa = Variable('xa', domain='int')
>>> ya = Variable('ya', domain='int')
>>> s = Forall([x, y, z],
...           Implies(And(Term('ParentOf', x, y),
...                       Term('Age', x, xa),
...                       Term('Age', y, ya)),
...                   Term('OlderThan', x, y)))

```

The domains should be either base types or defined types in the theory's `type_definitions` attribute.

Source code in src/typedlogic/datamodel.py

```

119 @dataclass
120 class Variable:
121     """
122     A variable in a logical sentence.
123
124     >>> x = Variable('x')
125     >>> y = Variable('y')
126     >>> s = Forall([x, y],
127     ...             Implies(Term('friend_of', x, y),
128     ...                     Term('friend_of', y, x)))
129
130     Variables can have domains (types) specified:
131
132     >>> x = Variable('x', domain='str')
133     >>> y = Variable('y', domain='str')
134     >>> z = Variable('z', domain='int')
135     >>> xa = Variable('xa', domain='int')
136     >>> ya = Variable('ya', domain='int')
137     >>> s = Forall([x, y, z],
138     ...             Implies(And(Term('ParentOf', x, y),
139     ...                         Term('Age', x, xa),
140     ...                         Term('Age', y, ya)),
141     ...                     Term('OlderThan', x, y)))
142
143     The domains should be either base types or defined types in the theory's `type_definitions` attribute.
144     """
145
146     name: str
147     domain: Optional[str] = None
148     constraints: Optional[List[str]] = None
149
150     def __eq__(self, other):
151         return isinstance(other, Variable) and self.name == other.name
152
153     def __str__(self):
154         return "?" + self.name
155
156     def __hash__(self):
157         return hash(self.name)
158
159     def as_sexpr(self) -> SExpression:
160         sexpr = [type(self).__name__, self.name]
161         if self.domain:
162             return sexpr + [self.domain]
163         else:
164             return sexpr
165
166     @staticmethod
167     def create(names: str) -> tuple[Variable, ...]:
168         return tuple(Variable(name.strip()) for name in names.split())
169

```

2.3.3 Sentence

Bases: ABC

Base class for logical sentences.

Do not use this class directly; use one of the subclasses instead.

Model:

```

classDiagram
    Sentence <|-- Term
    Sentence <|-- BooleanSentence
    Sentence <|-- QuantifiedSentence
    Sentence <|-- Extension

```

 **Source code in** `src/typedlogic/datamodel.py` 

```

172 class Sentence(ABC):
173     """
174     Base class for logical sentences.
175
176     Do not use this class directly; use one of the subclasses instead.
177
178     Model:
179
180     ```mermaid
181     classDiagram
182     Sentence <|-- Term
183     Sentence <|-- BooleanSentence
184     Sentence <|-- QuantifiedSentence
185     Sentence <|-- Extension
186     """
187
188     def __init__(self):
189         self._annotations = {}
190
191     def __and__(self, other):
192         return And(self, other)
193
194     def __or__(self, other):
195         return Or(self, other)
196
197     def __invert__(self):
198         return Not(self)
199
200     def __sub__(self):
201         return NegationAsFailure(self)
202
203     def __rshift__(self, other):
204         return Implies(self, other)
205
206     def __lshift__(self, other):
207         return Implied(self, other)
208
209     def __xor__(self, other):
210         return Xor(self, other)
211
212     def iff(self, other):
213         return Iff(self, other)
214
215     def __lt__(self, other):
216         return Term(operator.lt.__name__, self, other)
217
218     def __le__(self, other):
219         return Term(operator.le.__name__, self, other)
220
221     def __gt__(self, other):
222         return Term(operator.gt.__name__, self, other)
223
224     def __ge__(self, other):
225         return Term(operator.ge.__name__, self, other)
226
227     def __add__(self, other):
228         return Term(operator.add.__name__, self, other)
229
230     def __mul__(self, other):
231         # TODO: avoid circular import
232         from typedlogic.extensions.probablilistic import Probability, That
233         return Probability(self, That(other))
234
235     @property
236     def annotations(self) -> Dict[str, Any]:
237         """
238         Annotations for the sentence.
239
240         Annotations are always logically silent, but can be used to store metadata or other information.
241
242         :return:
243         """
244         return self._annotations or {}
245
246     def add_annotation(self, key: str, value: Any):
247         """
248         Add an annotation to the sentence
249
250         :param key:
251         :param value:
252         :return:
253         """
254         if not self._annotations:
255             self._annotations = {}
256         self._annotations[key] = value
257
258     def as_sexpr(self) -> SExpression:
259         raise NotImplementedError(f"type = {type(self)} // {self}")
260
261     @property
262     def arguments(self) -> List[Any]:
263         raise NotImplementedError(f"type = {type(self)} // {self}")
264
265
266
267

```

```

257
258
259
260
261
262
263
264
265
266

```

annotations property

Annotations for the sentence.

Annotations are always logically silent, but can be used to store metadata or other information.

Returns:

Type	Description
<code>Dict[str, Any]</code>	

add_annotation(key, value)

Add an annotation to the sentence

Parameters:

Name	Type	Description	Default
<code>key</code>	<code>str</code>		<i>required</i>
<code>value</code>	<code>Any</code>		<i>required</i>

Returns:

Type	Description

Source code in `src/typedlogic/datamodel.py` 

```

249 def add_annotation(self, key: str, value: Any):
250     """
251     Add an annotation to the sentence
252
253     :param key:
254     :param value:
255     :return:
256     """
257     if not self._annotations:
258         self._annotations = {}
259     self._annotations[key] = value

```

2.3.4 Term

Bases: [Sentence](#)

An atomic part of a sentence.

A ground term is a term with no variables:

```

>>> t = Term('FriendOf', 'Alice', 'Bob')
>>> t
FriendOf(Alice, Bob)
>>> t.values
('Alice', 'Bob')

```

```
>>> t.is_ground
True
```

Keyword argument based initialization is also supported:

```
>>> t = Term('FriendOf', dict(about='Alice', friend='Bob'))
>>> t.values
('Alice', 'Bob')
>>> t.positional
False
```

Mappings:

- Corresponds to AtomicSentence in Common Logic

 **Source code in** `src/typedlogic/datamodel.py` 

```

291 class Term(Sentence):
292     """
293     An atomic part of a sentence.
294
295     A ground term is a term with no variables:
296
297         >>> t = Term('FriendOf', 'Alice', 'Bob')
298         >>> t
299         FriendOf(Alice, Bob)
300         >>> t.values
301         ('Alice', 'Bob')
302         >>> t.is_ground
303         True
304
305     Keyword argument based initialization is also supported:
306
307         >>> t = Term('FriendOf', dict(about='Alice', friend='Bob'))
308         >>> t.values
309         ('Alice', 'Bob')
310         >>> t.positional
311         False
312
313     Mappings:
314
315     - Corresponds to AtomicSentence in Common Logic
316     """
317
318     def __init__(self, predicate: str, *args, **kwargs):
319         self.predicate = predicate
320         if not args:
321             self.positional = None
322             bindings = {}
323         elif len(args) == 1 and isinstance(args[0], dict):
324             bindings = args[0]
325             self.positional = False
326         else:
327             bindings = {f"arg{i}": arg for i, arg in enumerate(args)}
328             self.positional = True
329         self.bindings = bindings
330         self._annotations = kwargs
331
332     @property
333     def is_constant(self):
334         """
335         :return: True if the term is a constant (zero arguments)
336         """
337         return not self.bindings
338
339     @property
340     def is_ground(self):
341         """
342         :return: True if none of the arguments are variables
343         """
344         return not any(isinstance(v, Variable) for v in self.bindings.values())
345
346     @property
347     def values(self) -> Tuple[Any, ...]:
348         """
349         Representation of the arguments of the term as a fixed-position tuples
350         :return:
351         """
352         return tuple([v for v in self.bindings.values()])
353
354     @property
355     def variables(self) -> List[Variable]:
356         """
357         :return: All of the arguments that are variables
358         """
359         return [v for v in self.bindings.values() if isinstance(v, Variable)]
360
361     @property
362     def variable_names(self) -> List[str]:
363         return [v.name for v in self.bindings.values() if isinstance(v, Variable)]
364
365     def make_keyword_indexed(self, keywords: List[str]):
366         """
367         Convert positional arguments to keyword arguments
368         """
369         if self.positional:
370             self.bindings = {k: v for k, v in zip(keywords, self.bindings.values(), strict=False)}
371             self.positional = False
372
373     def __repr__(self):
374         if not self.bindings:
375             return f"{self.predicate}"
376         elif self.positional:
377             return f'{self.predicate}({",".join(f"{v}" for v in self.bindings.values())})'
378         else:
379             return f'{self.predicate}({",".join(f"{k}={v}" for k, v in self.bindings.items())})'
380
381     def __eq__(self, other):
382         # return isinstance(other, Term) and self.predicate == other.predicate and self.bindings == other.bindings
383         return isinstance(other, Term) and self.predicate == other.predicate and self.values == other.values
384
385     def __hash__(self):
386         return hash((self.predicate, tuple(self.values)))
387
388     def as_sexpr(self) -> SExpression:
389         return [self.predicate] + [as_sexpr(v) for v in self.bindings.values()]

```